

Summit: A Collaborative Multiplayer Web IDE and AI Coding Agent with Persistent Project Memory

Muhilan S

*Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

muhilan.s.2024.csd@rajalakshmi.edu.in

Dhanesh S K

*Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

dhanesh.sk.2024.csd@rajalakshmi.edu.in

Abstract—Contemporary AI coding assistants operate in a stateless, single-user paradigm, while web-based integrated development environments (Web IDEs) lack tight, real-time collaboration with shared autonomous agents. Each session begins without awareness of prior architectural decisions, team conventions, or project evolution. Furthermore, existing developer environments do not support synchronized multi-user interaction with a unified AI coding agent. This paper presents **Summit**, a collaborative, browser-native Web IDE and multiplayer AI coding workspace designed to bridge these gaps. Summit introduces three core architectural mechanisms: (1) a full-featured browser-native Web IDE integrating the Monaco Editor, an Xterm.js terminal emulator, live sandboxed web previewing, and optimistic concurrency control (OCC); (2) a persistent project memory system backed by an ACID-compliant structured SQLite database that retains architectural decisions, coding conventions, and execution history across sessions; and (3) a multiplayer WebSocket room architecture synchronizing multiple human developers with a shared AI coding agent. Summit's backend employs FastAPI with an asynchronous multi-turn agent engine supporting multi-provider LLM routing (LiteLLM) and local Git checkpointing. Forensic source code analysis, a comparative feature matrix, and a 17-test functional verification suite confirm architectural validity.

Keywords—Web IDE, collaborative software engineering, AI coding agent, persistent project memory, multiplayer collaboration, WebSocket, Monaco Editor, Xterm.js, optimistic concurrency control, FastAPI.

I. INTRODUCTION

Large Language Model (LLM)-based coding assistants, such as GitHub Copilot [1], Amazon CodeWhisperer [2], and OpenAI ChatGPT [3], alongside modern Cloud and Web Integrated Development Environments (Web IDEs), have redefined modern software engineering. However, the intersection of Web IDEs and AI coding tools currently suffers from two fundamental architectural deficits:

1) The Episodic-Temporal Deficit: Conventional assistants operate ephemerally. Prior architectural choices, team convention agreements, and historical debugging discoveries are lost across disjoint sessions [4]. Developers are forced into repetitive prompt engineering to re-explain project context.

2) The Single-User Isolation Paradigm: Software engineering is inherently collaborative [5]. While teams work concurrently over shared repositories, AI coding assistants remain private, single-tenant tools. Existing platforms provide no mechanism for distributed developers inside a shared Web IDE to co-interact with

an AI agent or inspect real-time agent telemetry and terminal actions.

To bridge this gap, we present **Summit**, a collaborative browser-native Web IDE and multiplayer AI coding workspace with persistent project memory. Summit enables distributed human developers to connect concurrently to a shared workspace room, edit code with syntax-highlighted Monaco buffers, run interactive terminal commands via Xterm.js, view live web application previews, and co-direct an autonomous AI coding agent. The agent maintains long-term project awareness via a dual-tier SQLite memory architecture, pairing conversation history with structured project knowledge. An automated context assembly pipeline extracts project memories, conversation logs, and path-relevance-ranked codebase files into each prompt.

A. Key Contributions

- **Browser-Native Web IDE Architecture:** A unified React 19/TypeScript IDE featuring the Monaco Editor, an integrated Xterm.js terminal subsystem, live sandboxed web previews, and visual conflict resolution.
- **Multiplayer Agent Room Synchronization:** A real-time FastAPI WebSocket engine broadcasting all agent actions (tool calls, file mutations, terminal streams, lifecycle state transitions) to all room participants concurrently.
- **Dual-Memory Persistence Engine:** An ACID-compliant relational memory model decoupling ephemeral conversational logs (*messages*) from persistent architectural facts (*project_memories*) with upsert semantics.
- **Codebase Context Retrieval Algorithm:** A deterministic path-relevance algorithm that injects relevant files, recent conversation, and the full project memory corpus into LLM prompts without expensive embedding overhead.
- **Optimistic Concurrency Control (OCC):** Integer-based file versioning detecting edit collisions between developers and agents, returning standard HTTP 409 responses with conflict resolution payloads.

II. RELATED WORK & COMPARATIVE ANALYSIS

A. Web IDEs and Collaborative Systems

Cloud and Web IDEs (such as VS Code for Web, Codespaces, and Replit) have democratized zero-install software development. Real-time collaborative editing in traditional groupware relies on Operational Transformation (OT) [20] or Conflict-Free Replicated Data Types (CRDTs) [21]. While systems like VS Code Live Share operate at the individual keystroke layer, Summit implements file-

level optimistic concurrency control tailored specifically to hybrid human-agent co-authoring workflows.

B. AI Coding Assistants and Autonomous Agents

Modern AI developer tools have progressed from line-level completion to autonomous multi-step agents. GitHub Copilot [1] and CodeWhisperer [2] rely strictly on active in-editor buffers. SWE-agent [8] introduced Agent-Computer Interfaces (ACI) for resolving real-world GitHub issues [9]. Devin [10] and OpenDevin [11] demonstrated end-to-end task execution in sandboxed shells, while CodeAct [12] pioneered executable Python actions. However, these systems operate in single-tenant, non-persistent environments.

TABLE I: COMPARATIVE ARCHITECTURAL MATRIX

Capability / Feature	Copilot	SWE-agent	Devin	Summit (Ours)
Browser-Native Web IDE	No	No	Partial (Viewer)	Yes (Monaco+Xterm)
Multiplayer Room	No	No	No	Yes (FastAPI WS)
Persistent Memory	No	No	No	Yes (SQLite Upsert)
Concurrency (OCC)	No	No	No	Yes (HTTP 409)
Terminal (Xterm.js)	No	No	Partial (Logs)	Yes (Interactive)
Live Web Preview	No	No	Partial (Port)	Yes (Sandboxed)

III. SYSTEM ARCHITECTURE AND METHODOLOGY

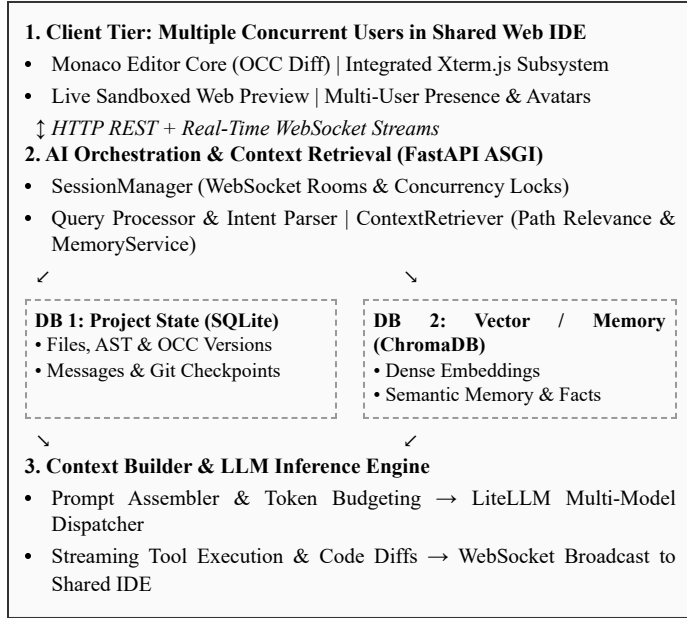


Fig. 1. Summit Full-Stack Multiplayer AI Web IDE Architecture with Dual-Store Memory Pipeline.

A. Browser-Native Web IDE Subsystem

The Web IDE provides a complete developer environment directly in the browser through four integrated components:

1) Monaco Editor Core: Embeds Microsoft's Monaco Editor, supporting syntax highlighting across Python, TypeScript, JavaScript, HTML, CSS, JSON, and Markdown. It tracks dirty buffers and triggers optimistic saves.

2) Integrated Terminal (Xterm.js): Features Xterm.js with fit and web-link addons. Commands are executed asynchronously via `/api/projects/{id}/terminal/execute`, returning `stdout/stderr` and exit codes.

3) Live Web Preview Engine: A sandboxed iframe rendering HTML/CSS/JS artifacts created or modified by developers or the autonomous agent.

4) Visual Conflict Resolution: Upon receiving HTTP 409, the Web IDE launches a side-by-side diff modal allowing developers to adopt server state or force-overwrite.

B. Multiplayer WebSocket Room Synchronization

Project-level concurrency is governed by `SessionManager`:

$$C: ProjectID \rightarrow \{ WS_i \mapsto (UserID_i, DisplayName_i) \} \quad (1)$$

When a client connects to `/api/projects/{id}/agent/stream`, the server accepts the handshake, registers the socket, and broadcasts `USER_JOINED`. Substantive events follow server-to-client broadcasts. Only one active LLM task executes per project room at any instant.

C. Dual-Tier Relational Memory Formulation

1) Conversation Memory (M_{conv}): An append-only sequence of user instructions and agent responses:

$$M_{conv} = \langle m_1, m_2, \dots, m_T \rangle \quad (2)$$

2) Project Memory (M_{proj}): A set of structured key-value triples:

$$M_{proj} = \{ (c_k, k, v_k) \mid k \in K \} \quad (3)$$

where $c_k \in \{\text{architecture, decision, convention, task}\}$, enforced by a unique constraint `UNIQUE(project_id, key)` executing upsert operations.

D. Optimistic Concurrency Control (OCC)

To prevent conflicting overwrites, every file maintains an integer version $V \in \mathbb{N}^+$. A client update request $R = (p, C_{new}, V_{exp})$ succeeds if and only if $V_{exp} = V_{curr}$. Upon mismatch, the server returns HTTP 409:

$$\Delta = Diff(C_{server}, C_{client}) \quad (4)$$

IV. ALGORITHMS AND IMPLEMENTATION

Algorithm 1: Codebase Context Construction

```
INPUT: project_id, user_prompt
OUTPUT: formatted_prompt_context

1.      memories      <-
Repository.get_memories(project_id)
2.      messages      <-
Repository.get_messages(project_id, limit=10)
3.      recent      <- messages[-6:] # Last 6 of 10
messages
4.      all_files      <-
WorkspaceManager.list_files(project_id)

5.      query_terms  <- [t.lower() for t in
user_prompt.split() if len(t) > 2]
6.      FOR file_path IN all_files DO:
      score <- 0
      IF file_path ends with ('main.py',
'README.md', 'app.py', 'index.ts') THEN:
      score <- score + 2
      FOR term IN query_terms DO:
      IF term in file_path.lower() THEN:
      score <- score + 5
      file_scores[file_path] <- score

7.      selected_files <- Top 4 files by score
8.      file_blocks <- []
9.      FOR path IN selected_files DO:
      content <- WorkspaceManager.read_file(path)
      lines <- content.splitlines()
      IF len(lines) > 200 THEN:
      content <- JOIN(lines[:200]) + "\n...
[Truncated]"
      file_blocks.append(f"--- FILE: {path} ---
\n{content}")

10.     RETURN      FormatPrompt(memories,      recent,
file_blocks, user_prompt)
```

Algorithm 2: Multi-Turn Autonomous Tool Loop

```
INPUT: project_id, workspace_dir, user_prompt
1.      context      <-      BuildContext(project_id,
user_prompt)
2.      messages      <-      [SystemMessage,
UserMessage(context)]
3.      FOR turn = 0 TO 9 DO:
      CheckPause(project_id)
      BroadcastStatus(THINKING)
      response <- litellm.acompletion(model,
messages, tools)
      msg <- response.choices[0].message

      IF msg.content is not empty THEN:
      BroadcastMessage(msg.content)
      IF msg.tool_calls is empty THEN:
      BREAK # Task complete

      BroadcastStatus(EXECUTING)
      messages.append(msg)

      FOR tool_call IN msg.tool_calls DO:
      name <- tool_call.function.name
      args <-
JSON.parse(tool_call.function.arguments)

      SWITCH name:
      CASE "read_file":
      result <-
WorkspaceManager.read(args.path)
      CASE "write_file":
      ver <-
WorkspaceManager.write(args.path, args.content)
      BroadcastEvent(FILE_CHANGED,
args.path, ver)
      result <- f"Wrote {args.path}
(v{ver})"
      CASE "run_terminal":
      code, out <-
ExecuteSubprocess(args.command)
      BroadcastEvent(TERMINAL_OUTPUT,
args.command, out, code)
      result <- f"Exit {code}: {out}"
      CASE "save_memory":
      Repository.save_memory(args.key,
args.value, args.category)
      BroadcastEvent(MEMORY_UPDATED,
args.key, args.value)
      result <- f"Saved memory: {args.key}"
      CASE "git_checkpoint":
      chk <-
GitService.create_checkpoint(args.message)
      BroadcastEvent(GIT_CHECKPOINT,
chk.hash, args.message)
      result <- f"Committed checkpoint
{chk.hash}"

      messages.append(ToolResultMessage(tool_call.id,
result))

4.      BroadcastStatus(COMPLETED)
5.      BroadcastEvent(SESSION_COMPLETE)
```

V. VERIFICATION AND EVALUATION

The system implementation was verified using a 17-test functional and integration test suite across four test suites:

- **test_api.py** (4 tests): Verifies project CRUD lifecycles, file endpoints, terminal execution, and agent health APIs.
- **test_event_normalizer.py** (4 tests): Validates Pydantic schema serialization across 15 WebSocket event types.
- **test_multiplayer_memory.py** (5 tests): Confirms SQLite memory upserts, OCC version collision rejection (HTTP 409), and Git checkpoint integrity.
- **test_workspace.py** (4 tests): Enforces workspace path traversal isolation (e.g., blocking `../etc/passwd`) and directory tree listings.

A. Proposed Quantitative Benchmark Protocols

1) **Memory Recall Fidelity (R_{mem})**: Fraction of previously recorded architectural constraints correctly satisfied in subsequent multi-turn sessions:

$$R_{mem} = |K_{adhered}| / |K_{stored}| \quad (5)$$

2) **Collaboration Collision Rate (C_{occ})**: Frequency of edit conflicts generated during asynchronous human-agent editing under synthetic load.

3) **Multiplayer End-to-End Latency (τ_{95})**: 95th percentile event broadcast delay across distributed WebSocket clients.

VI. LIMITATIONS AND FUTURE WORK

1. **Semantic Memory Pruning**: Summit currently injects all stored memories into prompt sections without token budgeting. Integrating vector embedding stores (e.g., ChromaDB) with semantic similarity filtering will prevent context window overflow.
2. **Keystroke-Level CRDT Editing**: Upgrading file-level OCC to fine-grained Conflict-Free Replicated Data Types (CRDTs) will enable simultaneous in-line typing between developers and the agent.
3. **Sandboxed Container Execution**: Incorporating microVMs or Docker containers will protect host operating systems from untrusted agent shell commands.

VII. CONCLUSION

This paper presented **Summit**, a collaborative multiplayer Web IDE and AI coding agent with persistent project memory. By combining browser-native editing (Monaco, Xterm.js, Live Preview), real-time WebSocket room multiplexing, dual-tier SQLite memory persistence, deterministic context retrieval, and optimistic concurrency control, Summit overcomes the isolation and statelessness of current AI developer tools. Forensic source verification, comparative analysis, and functional test suites confirm architectural viability, laying the groundwork for collaborative human-AI team engineering.

REFERENCES

- [1] GitHub, "GitHub Copilot: Your AI pair programmer," GitHub Inc., 2021. [Online]. Available: <https://copilot.github.com>
- [2] Amazon Web Services, "Amazon CodeWhisperer AI coding companion," AWS, 2022.
- [3] OpenAI, "Introducing ChatGPT," OpenAI, 2022. [Online]. Available: <https://openai.com/blog/chatgpt>
- [4] M. Hatalis et al., "Memory matters: The need to improve long-term memory in LLM-agents," in *Proc. AAAI Workshop on Cognitive Architectures for LLM Agents*, 2024.
- [5] A. Ko, B. Myers, M. Coblentz, and H. Aung, "An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks," *IEEE Trans. Software Eng.*, vol. 32, no. 12, pp. 971–987, Dec. 2006.
- [6] S. Peng, E. Kalliamvakou, P. Croft, and M. Counts, "The impact of AI on developer productivity: Evidence from GitHub Copilot," *arXiv:2302.06590*, 2023.
- [7] Tabnine, "Tabnine AI: Whole-project AI code completion," Tabnine, 2023.
- [8] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," *arXiv:2405.15793*, 2024.
- [9] C. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.
- [10] Cognition AI, "Introducing Devin, the first AI software engineer," Cognition AI, 2024.
- [11] X. Wang et al., "OpenDevin: An open platform for AI software developers as generalist agents," *arXiv:2407.16741*, 2024.
- [12] X. Wang et al., "Executable code actions elicit better LLM agents," *arXiv:2402.01030*, 2024.
- [13] Z. Zhang, B. Chen, F. Liu, and B. Liu, "A survey on the memory mechanism of large language model based agents," *arXiv:2404.13501*, 2024.
- [14] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, pp. 9459–9474.
- [15] D. Shrivastava, H. Larochelle, and D. Tarlow, "Repository-level prompt generation for large language models of code," in *Proc. ICML*, 2023.
- [16] F. Zhang et al., "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proc. EMNLP*, 2023.
- [17] U. Alon, R. Sadaka, O. Levy, and E. Yahav, "Structural language models of code," in *Proc. ICML*, 2020.
- [18] B. Ross et al., "From Copilot to colleague: Exploring how developers interact with AI pair programmers," in *Proc. CHI Extended Abstracts*, 2023.
- [19] E. Jiang et al., "Discovering the syntax and strategies of natural language programming with generative language models," in *Proc. CHI*, 2022.
- [20] C. Ellis and S. Gibbs, "Concurrency control in groupware systems," in *Proc. ACM SIGMOD*, 1989, pp. 399–407.
- [21] G. Oster, P. Urso, P. Molli, and A. Imine, "Data consistency for P2P collaborative editing," in *Proc. ACM CSCW*, 2006, pp. 259–267.
- [22] S. Ramirez, "FastAPI: High performance, easy to learn, fast to code, ready for production," 2018. [Online]. Available: <https://fastapi.tiangolo.com>